

REMON: Remote External Memory Over the Network

Shiquan Zhang, Michail Bachras, Yuqiu Zhang, Yunhao Mao, Hans-Arno Jacobsen

Department of Electrical and Computer Engineering

University of Toronto

Toronto, ON, Canada

{shiquan.zhang, michalis.bachras, quincy.zhang, yunhao.mao}@mail.utoronto.ca

jacobsen@eecg.toronto.edu

Abstract—Modern database systems face significant challenges in managing memory efficiently, especially under conditions of resource heterogeneity, constrained local memory, and growing dataset sizes. To address these issues, we introduce REMON, a lightweight, software-based disaggregated memory system that leverages remote memory resources over the standard networking stack. REMON employs page-level memory management and efficient swapping mechanisms, enabling seamless memory allocation, release, and swapping between compute and memory nodes. Across standard analytics benchmarks, REMON reduces average latency by up to 34% over disk swapping under memory pressure and incurs less than 15% performance loss relative to fully in-memory execution when data fits in RAM. Compared to the RDMA-based remote-memory baseline, REMON is on average 11% slower but remains deployable and cost-effective on commodity cloud instances; its benefits are strongest in low-latency, high-bandwidth in-region clusters. With its user-space design over commodity TCP/IP and cloud-native compatibility, REMON offers a versatile, cost-effective memory extension for cloud databases.

Index Terms—cloud database, memory management, disaggregated memory

I. INTRODUCTION

The exponential growth of data generated by modern applications has dramatically increased the demand for efficient analytical processing systems capable of handling large datasets in-memory [1], [2]. Analytical workloads—common in domains such as business intelligence, scientific computing, and big data analytics—are predominantly read-intensive and require high-throughput, low-latency data access to deliver timely insights [3]–[5]. In-memory databases and data processing engines have emerged as essential tools for such tasks, offering significant performance advantages by minimizing data access latencies [6]–[9].

In cloud environments, which provide on-demand resource provisioning and scalability, deploying in-memory analytics presents unique challenges. Procuring servers with sufficient memory to hold entire large datasets can be prohibitively expensive. Instances with terabytes of RAM are available on public clouds but come at a higher cost compared to standard instances. For example, on AWS, the `r7iz.32xlarge` instance type equipped with 1TB memory incurs 40% higher cost than multiple `r5a.2xlarge` instances that have the

same amount of memory in aggregate [10]. These high-memory instances may also have limited availability across regions and configurations, restricting flexibility and scalability [11]. To address the challenge of single-instance memory constraints, a more cost-efficient approach is to process data using disk-based methods or to ingest data from object stores such as Amazon S3 [12]. However, the higher access latencies associated with disk I/O and remote object storage can degrade performance for read-intensive analytical workloads [13], [14]. Swapping to disk when memory is insufficient introduces additional overhead, further impacting system responsiveness and query execution times [15]–[17]. We make this positioning explicit by comparing against SSD-backed disk-swap baselines and quantifying the resulting cost-performance trade-offs in Section V.

Remote Direct Memory Access (RDMA) technologies have been proposed to mitigate these issues by enabling direct memory access between servers over the network with low latency and high throughput [18]. Systems like Infiniswap [19] and LegoOS [20] leverage RDMA to create remote memory pools, effectively expanding available memory resources. While RDMA offers impressive performance benefits, it requires specialized hardware and network infrastructure—such as RDMA-capable network interface cards (NICs) and loss-less networks—that are not widely available or cost-effective in commodity cloud environments. Although some cloud providers offer RDMA-enabled instances, they are often more expensive and less flexible than standard instances, limiting their practicality for dynamic, scalable deployments. According to AWS documentation [21], RDMA support is limited to specific instance types and requires placement within a single cluster, reducing the elasticity benefits of cloud computing. Other approaches like Clio [22] require customized hardware or networking stack modifications, which are also unavailable on public commodity clouds.

Emerging disaggregated memory architectures present a promising approach to address these challenges by decoupling compute and memory resources [23], [24]. In disaggregated architectures, memory and compute resources can be scaled independently and accessed over the network, allowing for greater flexibility and efficient resource utilization [22], [23]. This model aligns well with the elasticity of cloud environ-

ments, where resources can be provisioned instantaneously and adjusted dynamically based on workload demands. By embracing disaggregation, systems can leverage the collective memory resources of multiple nodes, effectively expanding memory capacity without the need for expensive high-memory servers. Distributed analytics engines and managed warehouses (e.g., Spark [8], Redshift [25], and Snowflake [26]) scale out not only memory but also compute and storage resources, but incur overhead such as distributed scheduling and data shuffle.

To harness the benefits of disaggregated architectures without the drawbacks of specialized hardware dependencies, we propose REMON (Remote External Memory Over the Network), a lightweight, user-space remote shared memory system. REMON expands a server’s available memory by aggregating memory resources across multiple cloud instances using the standard TCP/IP networking stack, avoiding the need for RDMA or other specialized technologies. By operating entirely in user space and relying on ubiquitous networking protocols, REMON offers wide applicability and ease of deployment in commodity cloud environments.

We integrate REMON with DuckDB [27], an in-process analytical SQL database management system designed for efficient query execution on modern hardware. DuckDB provides robust support for analytical workloads and is optimized for single-node execution, making it an ideal candidate for our experiments. By backing DuckDB with REMON as a transparent memory-extension layer through a narrow interface, we enable the processing of large analytical workloads that exceed the memory capacity of individual servers without incurring the high costs associated with large-memory instances or the performance penalties of disk-based methods. Our approach allows data to be ingested from attached object stores or disks and processed in a distributed shared memory environment, leveraging the elasticity and scalability of cloud resources. We conduct evaluations on DuckDB with REMON using TPC-H [28], TPC-DS [29], and Star Schema Benchmark (SSB) [30]. Our evaluation shows that REMON achieves up to 34% lower latency than disk swapping under memory pressure and incurs less than 15% overhead relative to fully in-memory execution when data fits in RAM.

To summarize, our contributions in this paper are as follows:

- 1) We introduce REMON, a remote shared memory system that enables disaggregated memory architectures using standard networking protocols in cloud environments.
- 2) Through detailed discussions on the architecture and implementation, we demonstrate REMON’s design principles and its integration with DuckDB to facilitate in-memory analytical processing of large datasets.
- 3) Using realistic analytics benchmarks, we evaluate REMON against state-of-the-art alternatives, including disk-based spill/swap baselines, RDMA-based remote memory solutions, and distributed analytical databases. We further evaluate REMON on AWS to showcase its deployability and analyze its cost-performance trade-offs on commodity clouds.

- 4) We discuss the implications of our findings for the design of scalable, cost-effective analytical processing systems in cloud environments and outline potential directions for future research.

The remainder of this paper is structured as follows: Section II reviews related work in remote memory systems and disaggregated architectures. Section III and IV describe the architecture and implementation of REMON, respectively, and its integration with DuckDB. Section V presents our experimental evaluation and results. Section VI discusses the broader implications of our work and potential future enhancements. Section VII concludes the paper.

II. RELATED WORK

The proliferation of data-intensive applications has encouraged extensive research into efficient memory management and data processing architectures. In this section, we review prior work related to our study, focusing on distributed shared memory systems, disaggregated memory architectures, and remote memory solutions. We differentiate our approach from existing methods and highlight how REMON advances the state of the art.

Distributed Shared Memory Systems. Early distributed shared memory (DSM) systems, such as TreadMarks [31], provided a shared memory abstraction over networks of workstations. TreadMarks employed lazy release consistency [32] to reduce coherence overhead by propagating memory updates lazily at synchronization points. Similarly, Li and Hudak’s work on shared virtual memory [33] addressed memory coherence in distributed environments. However, these systems were designed for tightly coupled clusters with homogeneous networks and were not optimized for the high latencies and variability of cloud environments. REMON differs by targeting commodity cloud infrastructures without relying on specialized cluster conditions, focusing on read-intensive analytical workloads where strict coherence is less critical.

Memory Disaggregation with Specialized Hardware. Recent studies like the work by Wang et al. [34] advocate for RDMA-enabled memory disaggregation to improve performance in distributed databases. These solutions rely on RDMA hardware to reduce network latency for remote memory access. Systems such as ScaleStore [35] also leverage RDMA and customizations to provide high performance in disaggregated settings; in contrast, REMON targets deployability on commodity clouds without RDMA. Other approaches like Clio [22] propose an amalgamation of customized memory systems, network systems, and compute offloading mechanisms. However, these customized hardware solutions are not widely available or cost-effective in cloud environments. REMON contrasts with these approaches by operating on public general-purpose clouds over standard networking protocols, eliminating the need for specialized hardware and making it more applicable to a broader range of cloud deployments.

Software-Based Remote Memory Solutions. Software approaches such as Infiniswap [19] and Fastswap [36] provide

remote memory access without requiring application modifications. Infiniswap leverages RDMA for low-latency swapping, while Fastswap optimizes swapping in virtual machines through kernel modifications. PolarDB [37] also uses a customized kernel to manage memory across nodes. These methods introduce complexity due to kernel-level changes and are less flexible in cloud environments where kernel access may be restricted. In contrast, REMON operates entirely in user space, avoiding kernel dependencies and simplifying deployment in cloud settings. Compared to RDMA-centric systems and kernel-level designs, REMON’s user-space approach targets broad deployability in commodity clouds without specialized NICs or OS changes.

Remote Regions [38] provide an OS/hypervisor-level abstraction for remote memory, enabling applications to allocate regions backed by remote DRAM; however, they integrate with the virtualization stack and require privileged changes. AIFM [39] integrates far memory directly into the application, achieving high performance via application-level control and specialized networking, but it requires application modifications and tighter coupling to the execution engine. By contrast, REMON remains entirely in user space and aims to minimize changes to existing applications while leveraging standard TCP/IP.

R3map [40] is a software-based system that enables applications to use remote memory via file-backed memory mapping over standard networks. By leveraging commodity hardware and focusing on efficient memory access, R3map supports dynamic chunk fetching, live migration, and adaptation to diverse network conditions. Its simplicity and user-space design make it a practical baseline for comparing REMON’s advanced capabilities in disaggregated memory management and scalability.

Disaggregated Memory Architectures. Disaggregated architectures aim to separate compute and memory resources for independent scaling and efficient utilization. RAMCloud [41] aggregates cluster DRAM and exposes a low-latency storage service via RPC. While it delivers high performance at cluster scale—typically leveraging high-speed interconnects (e.g., InfiniBand/RDMA) to reach microsecond-scale RPC latency—it is not an application-transparent remote-memory substrate in the paging sense. Our work instead focuses on user-space memory extension over standard TCP/IP. LegoOS [20] is an operating system designed for hardware resource disaggregation but requires significant changes to the OS and hardware support. Shoal [42] proposes a network architecture to support memory disaggregation but depends on specialized network designs. REMON differentiates itself by providing a disaggregated memory solution that works with existing cloud infrastructure, utilizing the inherent elasticity of the cloud to dynamically scale memory resources without specialized OS or network support.

In summary, while prior work has made significant progress in remote memory access and disaggregated architectures, many solutions depend on specialized hardware, kernel modifications, or custom infrastructures not readily available in

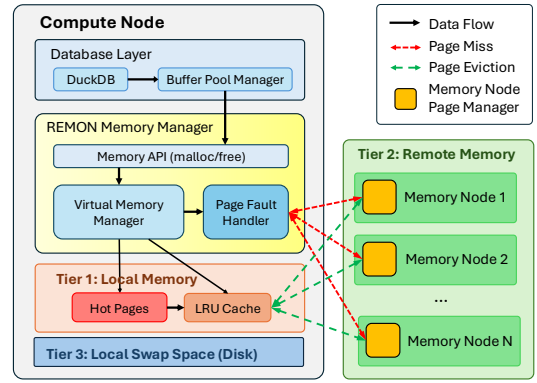


Fig. 1. System overview

commodity clouds. REMON fills this gap by offering a hardware-agnostic, user-space remote memory system that leverages standard networking, making it suitable for cost-effective deployment in cloud environments to support large-scale analytical workloads.

III. ARCHITECTURE

REMON enables memory expansion for applications such as database servers by leveraging remote memory resources. It facilitates dynamic memory allocation, release, and swapping between the compute node and memory nodes. This section describes the architectural components of REMON, including its memory organization, memory management strategies, page-tracking mechanisms, and memory management protocols.

A. System Overview

REMON implements a disaggregated memory architecture that enables applications to access a large virtual memory space spanning multiple physical servers. A key distinguishing feature of REMON is that its implementation lies entirely in user space, avoiding kernel modifications or specialized hardware requirements. This design choice ensures easy deployment and maintenance in existing cloud environments.

The system consists of two primary components: a Compute Node that runs the client application and manages memory access, and multiple Memory Nodes that provide additional memory capacity over the network. All memory management operations, including page fault handling and memory movement, are implemented through careful memory protection and signal handling mechanisms.

In typical deployments, multiple applications and compute nodes can concurrently use one or more shared memory nodes; REMON’s protocol maintains per-client sessions with per-session quotas to support concurrent multi-client operation over commodity networks.

Figure 1 illustrates REMON’s architecture. Client applications link against the REMON library on the compute node, which exposes standard memory APIs (e.g., malloc/free) while managing pages across local RAM and remote memory nodes. Under memory pressure, REMON retains hot pages locally and evicts colder pages selected by the LRU policy

to remote memory. When execution later accesses a non-resident page, REMON transparently fetches it back from the corresponding memory node and resumes execution. Tier 1 (Local Memory) denotes ordinary process pages currently residing in the compute node's RAM. REMON tracks the hot pages and applies LRU eviction in user space. The OS kernel is unmodified and only provides standard address-space management and fault delivery.

The design emphasizes three key principles: transparent memory expansion, efficient page movement using network-aware protocols, and hardware-agnostic operation using standard TCP/IP networking. REMON realizes these principles by coupling user-space virtual-memory control on the compute node with a lightweight page service on memory nodes. The compute node reserves and protects a managed address range so accesses to non-resident pages trigger a user-level fault handler, which consults per-page state/location metadata and fetches pages over persistent TCP connections. This approach also enables REMON to provide sophisticated memory management capabilities while maintaining broad compatibility across different environments and ensuring straightforward integration with existing systems.

B. Memory Organization

REMON organizes memory into fixed-size pages that form the basic unit of memory management. The system maintains a unified view of memory across three tiers:

- 1) **Local Memory:** Active pages in the Compute Node's RAM
- 2) **Remote Memory:** Pages stored in the Memory Nodes' RAM
- 3) **Local Swap Space:** Optional disk-based storage for custom page handling policies

While REMON primarily manages memory between local memory and remote memory nodes, it includes a local disk-based fallback mechanism (local swap space) that is activated only under specific conditions: (1) Remote memory nodes are unavailable or unreachable; (2) Remote memory resources are exhausted; (3) Network conditions make remote memory access inefficient. These conditions are detected via per-session capacity checks and connection error/time-out monitoring; when triggered, evictions are redirected to disk until remote capacity/connectivity recovers.

REMON manages data at page granularity. For allocations up to one page, REMON packs multiple objects into a page and tracks them as sub-page blocks via a per-page free list. For larger allocations, REMON reserves a contiguous range of whole pages as a page group and manages the allocation at the page level. In both cases, REMON maintains per-page metadata on the compute node to track each page's allocation and residency state. Section IV details the metadata layout and data structures for both strategies.

This dual-use organization supports efficient management for both small and large memory allocations, ensuring flexibility, reduced fragmentation, and optimized performance across varied allocation sizes.

1) *Page States:* To manage these diverse memory allocation patterns and facilitate movement between memory tiers, REMON tracks each page through a state system. These states reflect both the page's current location in the memory hierarchy and its operational status. Each page can exist in multiple states:

- **init:** The page is in an initial state, meaning it has been created but not yet allocated or used. It is essentially in a reserved state, waiting to be initialized for use.
- **local:** The page is currently in local memory and is actively being used.
- **local pinned:** The page is in local memory and cannot be swapped out. This state is particularly important for database systems, where certain critical pages (such as index structures or frequently accessed metadata) must remain in local memory to maintain performance and consistency. The ability to pin pages in local memory allows database systems to optimize their memory usage by ensuring that performance-critical data structures are always readily available.
- **remote:** The page has been swapped out to a remote memory node. This state indicates that the page's contents are stored on a remote node, which can be accessed over a network.
- **disk:** The page has been swapped out to disk. This state indicates that the page's contents are stored on disk to free up local memory. Accessing this page will require reading it back from disk.
- **remote cache:** The page is in a remote cache state, meaning it has been prefetched from a remote node but is not yet in active use. This state is used to improve performance by preloading pages that are likely to be accessed soon.
- **disk cache:** The page is in a disk cache state, meaning it has been prefetched from disk but is not yet in active use. This state is used to improve performance by preloading pages that are likely to be accessed soon.

Typical transitions include: (1) a fault on a non-resident page promotes it from `remote` or `disk` through a pinned transient state into `local`; (2) under memory pressure, `local` pages are demoted to `remote` (or to `disk` in disk-swap mode); and (3) `local pinned` pages become eligible for demotion only after an explicit unpin. Cache states reclassify to active on the first access.

2) *Memory Access Optimization:* REMON employs mechanisms to optimize memory access patterns and improve performance.

Hot Pages: REMON maintains a set of hot pages using lightweight per-page access counters. Under memory pressure, pages with higher counters are preferentially retained in local memory and prioritized for fast access. The hot pages mechanism helps reduce latency for commonly accessed data by keeping it readily available in local memory, which is particularly beneficial for OLAP database workloads with locality of reference.

Page Replacement Policies: REMON supports custom page replacement policies to manage memory efficiently. The Least

Recently Used (LRU) cache [43] is one implementation of these policies, providing a systematic approach to page replacement decisions. The LRU policy tracks page access patterns and helps determine which pages should remain in local memory and which should be candidates for swapping to remote memory or disk. When memory pressure occurs, pages that have not been accessed for the longest time are selected for eviction first. This flexible policy framework allows REMON to adapt its memory management strategy based on workload requirements, with LRU being particularly effective for database workloads where access patterns significantly impact query performance.

These mechanisms work together to provide efficient memory management: hot pages ensure quick access to frequently used data, while customizable page replacement policies, like LRU, provide systematic approaches to page replacement based on actual usage patterns.

The allocator mechanisms used in REMON (e.g., size classes and free lists for small objects, and page-granular management for large objects) follow standard designs found in established memory allocators [44], [45]. REMON’s novelty lies in making these pages *removable* in user space: while allocators such as jemalloc [46] and dlmalloc [47] manage objects within a single process address space, REMON additionally tracks page residency and migrates pages between local and remote memory to transparently extend capacity.

Compared to RDMA/kernel-based approaches, REMON’s user-space page management incurs additional overhead, but enables deployment in general cloud environments. We further quantify this overhead in Section V-B by comparing DuckDB+REMON against vanilla DuckDB when the working set fits in local RAM.

C. Page Movement

REMON implements a page movement system that manages memory transfers between compute nodes and memory nodes through user-level page fault handling. Each managed page is assigned a unique identifier (page ID) within a compute-node session, and the compute node maintains a lightweight page table that maps each page ID to its current state and location metadata (e.g., local/remote/disk). When DuckDB accesses a non-resident page, REMON intercepts the page fault through a custom handler, identifies the required page location, initiates necessary transfers, and resumes execution once the page is fetched to local memory.

The system manages memory primarily through page state transitions between local, local pinned, and remote states. Each transition ensures data consistency while using customizable page handling policies, like LRU, to maintain access pattern information. This enables REMON to make intelligent decisions about which pages to keep in local memory versus moving to remote memory.

The page movement strategy adapts based on allocation size to optimize performance and resource utilization. For small allocations, the system can reorganize memory within pages, coalescing free blocks and maintaining segregated

lists to optimize space usage. Large allocations, which span multiple contiguous pages, require coordinated movement of entire page groups. REMON maintains atomicity during these operations while supporting efficient splitting and combining of page groups as needed.

During periods of memory pressure, REMON initiates background page eviction based on the page handling policy and page states. To minimize access latency, the system provides prefetching mechanisms to proactively move pages from remote memory based on predicted access patterns. Pages predicted to be needed soon enter a remote cache state, reducing latency for subsequent accesses while maintaining efficient use of both local and remote memory resources.

While REMON primarily targets remote memory as the spill destination, it also supports a disk-based spill path that writes evicted pages to local temporary files and fetches them back on demand. We use this path in our evaluation (Section V-B) to compare REMON against DuckDB’s buffer-pool spilling and OS swapping baselines under memory pressure.

For RDMA-based systems such as Infiniswap, they integrate remote memory into the OS swap mechanism by modifying the kernel to redirect swap I/O to remote RAM over RDMA networking, while the paging policy and page movement decisions remain controlled by the kernel memory manager. In contrast, REMON implements paging in user space and explicitly controls page movement using its own metadata and policies. Although our current policy is simple (e.g., LRU-based eviction), REMON can be extended to incorporate DBMS-aware placement and eviction policies that are difficult to express through the generic kernel paging system.

D. Memory Node Design and Protocol

The memory node architecture and its associated memory management protocol form the foundation of REMON’s disaggregated memory capabilities. A key distinguishing feature of REMON is its multi-tenant design: each memory node functions as a dedicated memory pool that can simultaneously extend the available memory of multiple compute nodes through a disaggregated architecture. This distributed system operates through a session-based protocol that provides *per-session quota isolation*: each compute node establishes a session with explicit capacity limits, and the memory node enforces admission and accounting within that quota. REMON’s multi-client support is intended for cooperative deployments where remote memory is sufficient to prevent accidental resource interference.

Memory management within memory nodes operates through a session-based protocol that begins with connection establishment. When a client initiates a connection, the memory node establishes a dedicated session and allocates resources for that client’s use. Each client session receives a unique identifier that allows the memory node to track resource utilization independently. This session-based approach enforces per-session quotas and avoids accidental resource interference (e.g., one client exhausting shared capacity) during normal operation. To avoid race conditions under concurrent

store/release requests, the memory node performs quota checks and accounting updates atomically, admitting a store request only if sufficient session quota remains.

The protocol supports three fundamental types of operations: memory storage, retrieval, and release. When DuckDB requires additional memory beyond local capacity, the protocol facilitates the transparent movement of pages to remote memory. The memory node’s management system performs per-session admission and accounting (quota enforcement) and maintains per-session page tables for tracking remote pages. For retrieval operations, the protocol restores pages to local memory; for release operations, it reclaims session-owned remote pages and updates accounting.

Resource management within memory nodes implements several optimization techniques to ensure efficient operation while enforcing per-session quotas. The node monitors per-session memory usage, manages page allocation with tenant-specific limits, and can implement local page-management policies to maintain utilization within each session’s budget. The protocol supports these optimizations through features such as persistent connections between nodes to reduce overhead and provisions for caching frequently accessed pages, reducing network traffic and access latency for common operation patterns.

REMON’s distributed architecture allows multiple memory nodes to be deployed in a system to increase total memory capacity and to distribute remote allocations across nodes. The protocol supports this scalability by implementing concurrent operations from multiple clients while enforcing per-session quotas.

IV. IMPLEMENTATION DETAILS

REMON implements its memory management functionality through several key components and operations. This section details the implementation of core functions and components in REMON, focusing on memory allocation strategies, state transitions in the memory hierarchy, and the design of asynchronous operations that ensure efficient interaction with disk and remote memory nodes.

REMON exposes a small user-space interface on compute nodes and a session-based protocol on memory nodes. These interfaces are described in the sections below:

- **Allocation.** `remon_malloc` and `remon_free` allocate and free objects from and to REMON-managed pages.
- **State transitions.** `remon_promote` and `remon_demote` implement page promotions on misses and demotions under memory pressure.
- **Forced operations.** `remon_force_swap`, `remon_force_unmap`, and range variants provide explicit control over page placement for large regions.

A. Memory Allocation Implementation

The memory allocation system in REMON distinguishes between small and large memory requests. The `remon_malloc` function dispatches requests up to the page size to a page-local allocator backed by size-class free lists, while

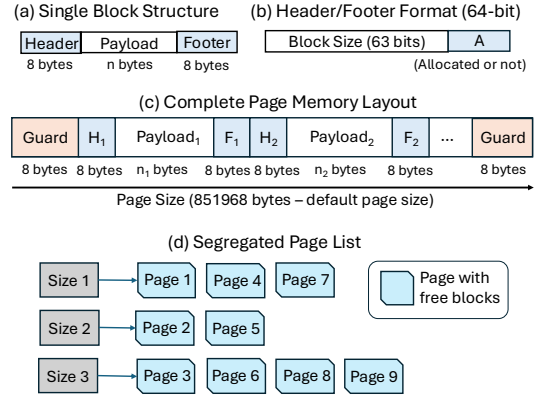


Fig. 2. Small memory allocation

larger requests are served by allocating contiguous page groups managed at page granularity.

The implementation uses boundary tags to support efficient coalescing during `remon_free`. For small allocations, each block stores a compact header/footer encoding size and allocation status, which enables coalescing adjacent free blocks while keeping per-page free lists consistent.

Figure 2 illustrates the organization of small memory allocations in REMON:

- Single Block Structure:** Shows the fundamental memory block layout with an 8-byte header, variable-sized payload area, and 8-byte footer, enabling efficient block management and coalescing.
- Header/Footer Format:** Details the 64-bit structure of boundary tags, using 63 bits for block size and 1 bit for allocation status.
- Complete Page Memory Layout:** Shows the internal structure of a memory page, including guard regions for memory access protection and the arrangement of memory blocks with their headers and footers.
- Segregated Page List:** Demonstrates how pages are organized by the maximum size of their free blocks. Each size class maintains a list of pages containing free blocks of that size, enabling quick allocation of appropriately sized memory blocks.

For large allocations, the top level allocator employs a size-indexed mapping system (`huge_page_map`) that tracks available page groups. Boundary tags are applied at the page level, with each large allocation consisting of a header page containing metadata such as `is_free` and `global_size`, and a tail page that duplicates this information for multi-page allocations. This allocator efficiently handles the splitting and coalescing of contiguous pages, while ensuring proper page alignment for optimal performance.

Figure 3 illustrates this organization. The `huge_page_map` maintains size classes for different page counts (2, 3, and 4 pages), where each size class tracks lists of contiguous free pages. Free pages of the same size class are organized in doubly-linked lists, enabling efficient coalescing and splitting. The figure also shows an example of a 3-page allocation, demonstrating how metadata is stored in both head and tail

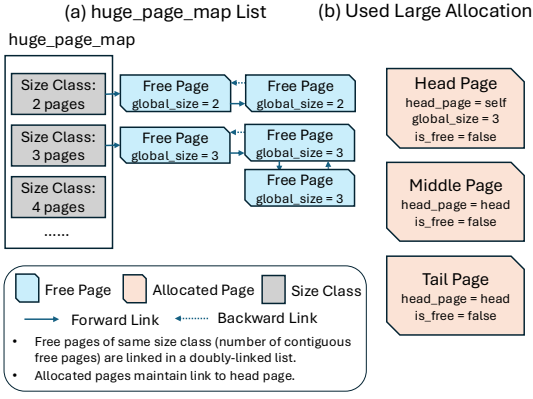


Fig. 3. Large memory allocation

pages, with the middle page linked between them. Each allocated page maintains its global size and free status, with head pages serving as the primary reference point for the entire allocation.

The boundary tag scheme, applied consistently at both block and page levels, enables efficient merging of freed memory across the entire system, regardless of allocation size. Both the free list allocator and the top level allocator work in concert with the page movement system to ensure efficient memory utilization. This design also supports multi-client operation through per-session tracking and quota enforcement at the memory nodes.

Memory deallocation, implemented in `remon_free`, releases memory based on the size of the original allocation. For smaller allocations, it works within individual pages to coalesce adjacent free blocks whenever possible, ensuring efficient memory reuse. For larger allocations that span multiple pages, it coordinates the release of page sequences while preserving memory consistency.

REMON guards page metadata updates (e.g., state transitions and location mappings) with standard synchronization primitives (e.g., mutexes) to prevent inconsistent state transitions under concurrent faults, evictions, and pin/unpin operations. At the memory-node side, quota checks and accounting updates are serialized per session, preventing one session from consuming another session's remote memory budget.

In a deployment with one compute node and multiple memory nodes, the compute node maintains a session with each memory node with a per-session quota. When memory pressure triggers demotion, REMON selects a target memory node based on available capacity and records the page's location in its page table. On a subsequent access, the fault handler consults this mapping, fetches the page from the recorded memory node, restores local access permissions, and resumes execution.

B. State Transition Operations

REMON implements memory state transitions through `remon_promote` and `remon_demote` functions. The `remon_promote` function manages the movement of memory closer to the CPU in the memory hierarchy. When invoked,

it evaluates the current state of the target memory region and initiates asynchronous data transfer from remote memory or disk storage if necessary. The function implements an efficient state machine that handles transitions between remote, disk, and local states while maintaining data consistency.

The `remon_demote` function implements the opposite transition, preparing memory for potential eviction to remote memory. It coordinates with the custom page replacement policy tracking system to update access patterns and adjusts memory protection settings accordingly. The implementation ensures that memory remains accessible while being marked as a candidate for eviction under memory pressure.

C. Forced Memory Operations

REMON implements explicit page movement control through forced operations that can provide DuckDB with direct control over page placement. The `remon_force_swap` function bypasses normal eviction by explicitly removing a page from the page replacement list and initiating asynchronous eviction to remote memory or disk storage.

The `remon_force_unmap` function provides a mechanism to unmap a set of memory-resident pages and reset memory regions to their initial state. This transition involves resetting the page's state, releasing its memory resources, and reinitializing its allocator.

Both operations support range-based variants (`remon_force_swap` and `remon_force_unmap_range`) that operate on multiple consecutive pages while allowing specific pages to be excluded from the operation. This range support enables efficient memory management during database operations that affect large, contiguous memory regions, such as table scans or large joins.

D. Integration with DuckDB

We integrated REMON with DuckDB version 0.8.1. In our implementation, DuckDB interacts with REMON primarily through the allocation interface: we redirect DuckDB allocator calls (i.e., `AllocateData`, `FreeData`, and `ReallocateData` in DuckDB's `Allocator` class) to REMON's allocation functions, `remon_malloc`, `remon_free`. The DuckDB buffer manager remains unmodified, viewing REMON pages as standard pointers. This integration pattern generalizes to other in-process analytics engines with a centralized allocator/buffer manager enabling memory capacity extension.

V. EVALUATION

We evaluated REMON's performance by integrating it with DuckDB, comprehensively across various configurations, focusing on its ability to manage large datasets efficiently and maintain optimal performance under different memory constraints. To benchmark REMON's capabilities, we compared against disk-swap-based baselines and included *Infiniswap* [19] and *R3map* [40] as remote-memory baselines, as well as *Spark SQL* [48] as a distributed database baseline. We

Configurations	Private Cloud	CloudLab	AWS
Cluster type	-	APT c6220	m5.4xlarge (comp) r5.xlarge (mem)
vCPU number	16 (comp) / 4 (mem)		
Memory size	64 GB		
Disk size	1 TB		
Disk type	SSD		
Operating system	Ubuntu 22.04.4 LTS		
Network bandwidth (Gbps)	9.91	TCP/IP: 9.40 RDMA: 46.07	in-region: 3.74 cross-region: 0.215
Raw network latency (ms)	0.55	TCP/IP: 0.16 RDMA: 1.09×10^{-3}	in-region: 0.75 cross-region: 58.6

TABLE I
EXPERIMENT SETUP

employed TPC-H [28], TPC-DS [29] and SSB [30] benchmarks, which emulate real-world, data-intensive operations in database systems. We also deployed REMON on AWS EC2 clusters and present a cost-performance comparison against baseline solutions in commodity cloud environments.

A. Experiment Setup

We conducted experiments on three platforms: a private cloud setup (most performance results), CloudLab [49] (RDMA-based baselines), and AWS EC2 (commodity-cloud deployment and cost-performance analysis). Table I summarizes the platform configurations. The evaluation setup was designed to assess REMON’s performance under different system configurations, focusing on the following variables:

Local Memory Limits: We varied the size of local RAM on the compute node, emulating different memory constraints to evaluate REMON’s ability to operate under limited local RAM conditions. In the evaluations on REMON, the system swap space was switched off.

Dataset Size: We scaled the datasets to different sizes to evaluate how REMON handles varying data volumes and the impact of dataset size on its memory management performance. The memory usage of TPC-H queries typically ranges from 1 to 2 times of the dataset size. For example, with a scale factor of 8 GB, memory usage can range from 8 GB to 16 GB, depending on the execution and intermediate results generated during query processing.

Swap-to-Disk (S2D) Variants. We considered three swap-to-disk mechanisms that arise at different layers of the stack: (1) *REMON S2D*: DuckDB runs on REMON and, with remote memory disabled in this configuration, REMON spills pages to local temporary files and later reads them back. (2) *DuckDB S2D*: vanilla DuckDB spills via its buffer manager under a constrained memory budget. (3) *OS S2D*: Vanilla DuckDB runs in an in-memory configuration and the operating system handles memory pressure via swapping. In our experiment setup, all disk-based spill/swap paths use local SSD storage.

Benchmarks: We evaluated REMON with DuckDB on three benchmarks: TPC-H, TPC-DS and SSB. TPC-H (22 queries) represents classic decision-support workloads with diverse join and aggregation patterns. TPC-DS (99 queries) is substantially more complex and stresses the engine with richer SQL constructs and heavier intermediate state, making it a strong test under memory pressure. SSB (13 queries) targets star-schema analytics with more regular access patterns and a

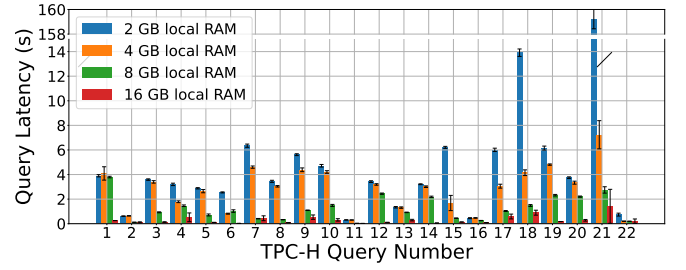


Fig. 4. REMON performance under varying local memory limits (SF = 8)

smaller footprint, complementing TPC-H/TPC-DS by isolating behavior on streamlined warehouse queries.

Each experiment was conducted five times, and the results are presented as averages with standard deviation bars to indicate variability.

B. Evaluation Results

Performance under Varying Memory Limits. For this set of evaluations, we fixed the TPC-H dataset scale factor to 8 GB and varied the local RAM on the compute node between 2 GB, 4 GB, 8 GB, and 16 GB. This allowed us to evaluate REMON’s ability to adapt to different memory configurations, particularly in environments with constrained local memory.

When the local RAM is smaller than the dataset size, REMON manages memory dynamically by offloading excess data to remote memory through page swaps over the network. As shown in Figure 4, REMON achieves near-optimal query runtimes when the local memory is sufficient to accommodate the dataset (e.g., 16 GB), as it eliminates the need for remote memory access. However, even under stricter memory constraints, REMON demonstrates robust performance by dynamically managing memory and efficiently swapping pages to remote memory nodes. For instance, with 8 GB of local memory, REMON incurs a modest runtime increase of approximately 15% compared to the fully in-memory case, while maintaining operational efficiency without excessive delays or failures.

In the most constrained scenario, with only 2 GB of local RAM, REMON supports execution despite frequent remote memory accesses. While runtimes increase compared to the fully in-memory 16 GB configuration, REMON ensures completion without compromising correctness, highlighting its suitability for memory-limited deployments.

Performance under Varying Dataset Sizes. To evaluate REMON’s performance under varying dataset sizes (scale factors), we kept the local RAM fixed at 8 GB and increased dataset sizes, ranging from 1 GB to 8 GB. This setup allowed us to assess how REMON manages memory as the dataset size grows.

As shown in Figure 5, REMON demonstrates its robustness and scalability as dataset sizes increase. For smaller datasets (1 GB and 2 GB), the entire dataset fits within local memory, allowing REMON to achieve near-optimal query runtimes. When dataset sizes grow beyond the local memory capacity (e.g., 4 GB and 8 GB), REMON seamlessly adapts by dynam-

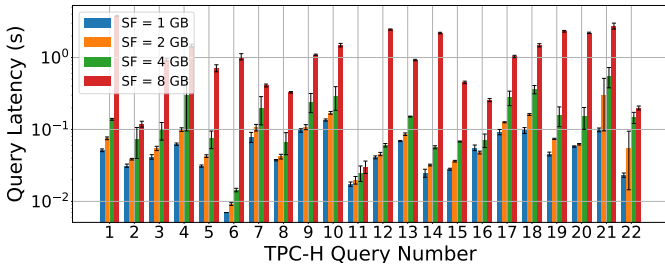


Fig. 5. REMON performance under varying scale factors (8 GB local RAM)

ically offloading excess data to remote memory. This ensures that queries continue to execute reliably, maintaining reasonable latencies even under constrained memory conditions.

For instance, when processing larger datasets with a scale factor of 8 GB, REMON enables memory-intensive queries like Query 18 and Query 21, which involve substantial intermediate results, to execute successfully. Even in scenarios with limited local memory, REMON ensures functionality and correctness, acting as a seamless extension of local resources.

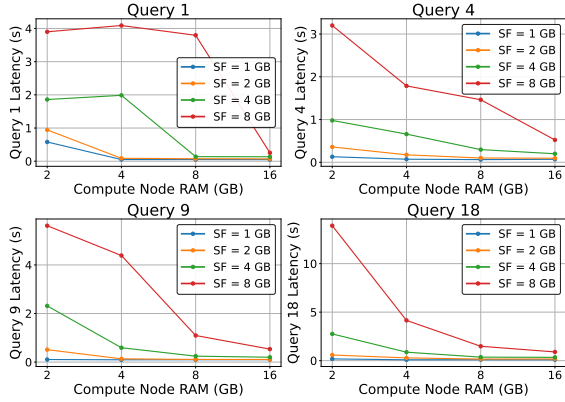


Fig. 6. REMON performance of Query 1, 4, 9 and 18

To provide a more detailed view of REMON’s performance under various configurations, Figure 6 presents query runtimes for representative queries (Q1, Q4, Q9, and Q18) across multiple scale factors (1, 2, 4, and 8 GB) and local RAM sizes (2, 4, 8, and 16 GB). The results highlight REMON’s adaptability: even as dataset sizes exceed local memory limits, REMON ensures that diverse queries—including basic scans (Query 1), read-heavy operations (Query 4), complex joins (Query 9), and memory-intensive tasks generating large intermediate results (Query 18)—execute efficiently. This demonstrates REMON’s capability to handle demanding workloads, maintaining strong performance and reliability in memory-constrained environments.

Handling Large-Scale Datasets. We ran TPC-H with a scale factor of 80 GB and varied compute-node RAM (100, 130, and 160 GB) to stress-test REMON as the working set approaches machine limits. These configurations reflect cloud deployments where datasets approach or exceed a single machine’s RAM capacity.

As shown in Figure 7, the four representative queries (Q7, Q13, Q18, and Q21) exhibit a consistent pattern: with 160

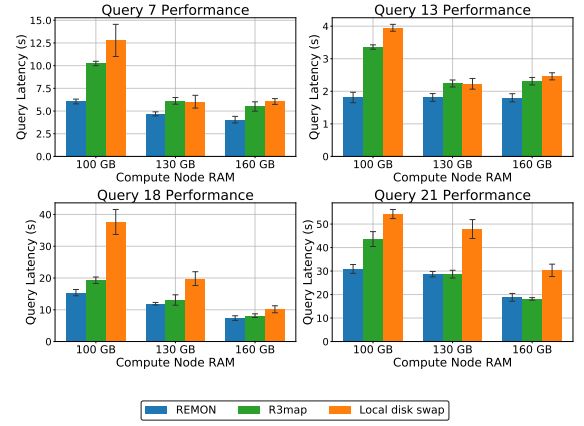


Fig. 7. REMON performance with large-scale datasets

GB, most working sets fit in memory and REMON’s latency is comparable to the two baselines.

At 130 GB and 100 GB, REMON’s user-space paging keeps hot pages local and demotes colder pages to remote memory. The additional network hops increase latency relative to the fully in-memory case, yet REMON remains faster than both baselines: R3map incurs synchronous remote page faults, while local disk swap is constrained by I/O contention, causing larger slowdowns under memory pressure.

Overall, REMON enables the compute node to process datasets near the size of local memory without severe degradation. By effectively balancing local memory use with remote memory offloading, REMON efficiently handles large-scale, data-intensive workloads even under constrained local RAM.

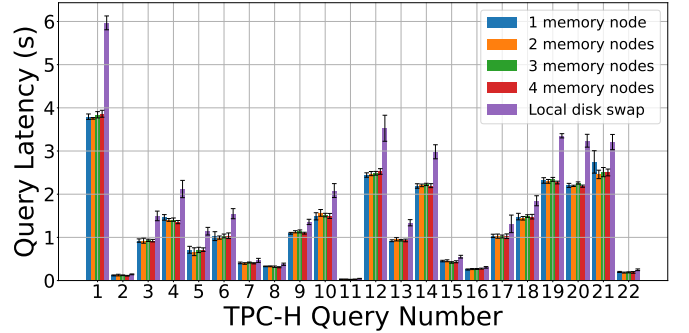


Fig. 8. REMON performance with varying number of memory nodes

Performance under Varying Number of Memory Nodes.

To assess the distribution overhead of partitioning the same remote memory budget across more memory nodes, we evaluated with local RAM of 8 GB, TPC-H scale factor of 8 GB, and a fixed 16 GB of remote memory split evenly across nodes. As shown in Figure 8, the average latencies remain stable as we add memory nodes under a fixed workload, indicating that REMON can distribute and retrieve pages across multiple memory nodes without introducing measurable overhead at this scale. This underscores REMON’s adaptability in cloud environments, where scalable memory resource allocation is essential.

Impact of Network Latency. In this set of experiments, we evaluated how network latency affects REMON’s perfor-

mance. Specifically, we introduced artificial network delays of 0 μ s, 100 μ s, 200 μ s, and 500 μ s using `netem` [50], with a 20% deviation. This setup emulates network conditions commonly observed within a cloud region [51]. We use these additional delay settings as a controlled sensitivity analysis. The median RTT measured in our AWS in-region (inter-AZ) deployment is 0.75 ms, providing an empirical reference for these settings. By comparing REMON’s performance under these conditions to the local disk swap baseline, we provide insights into how network speed influences remote memory access times and overall query execution.

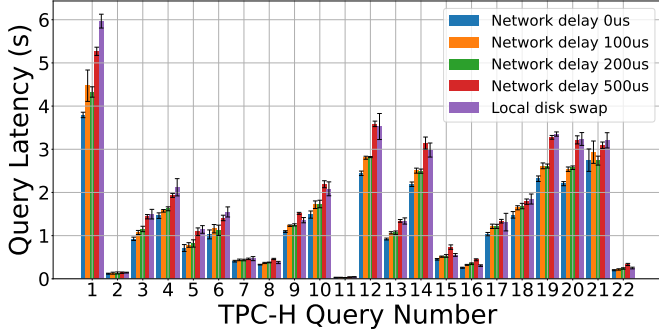


Fig. 9. REMON performance under varying network conditions

The compute node’s local RAM was fixed at 8 GB, matching the dataset size. As shown in Figure 9, the execution time for each query increases with higher network latencies. With an additional network latency of 500 μ s, query runtimes approach or even exceed those of the local disk swap baseline.

This result demonstrates that REMON’s performance is influenced by network conditions. When network latencies are comparable to those observed within a commodity cloud region, REMON with remote memory consistently outperforms local disk swap. However, as network delays increase, its advantage diminishes. Since REMON relies on network-based memory access, its performance is inherently tied to network conditions. These findings align with our deployment assumptions (same-region, low-latency, high-bandwidth links). The trade-offs associated with REMON’s design and its adaptability to various network environments are further discussed in Section VI.

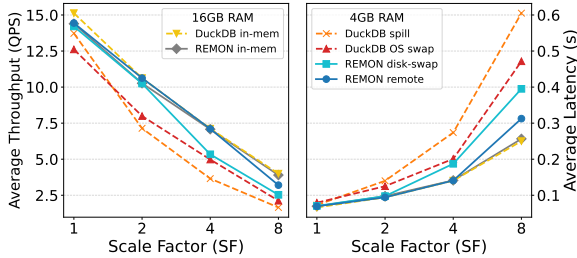


Fig. 10. REMON remote vs. swap-to-disk baselines (16 GB for in-memory baselines; 4 GB for memory-pressure settings)

REMON Overhead and S2D Baselines. Figure 10 demonstrates REMON’s overhead when the working set fits in local memory and its benefits under memory pressure compared to disk-based solutions.

For the in-memory case (16 GB local RAM), we ran vanilla DuckDB (*DuckDB in-mem*) and DuckDB with REMON enabled (*REMON in-mem*), where remote memory is not exercised because the working set fits. When memory fits, enabling REMON incurs low overhead: throughput decreases by 5.2%, and average latency increases by 3.8% relative to *DuckDB in-mem*.

For the memory-constrained case (4 GB local RAM), we compared *REMON remote* against three swap-to-disk baselines (detailed in Section V-A) across scale factors; all disk-based configurations use the same local SSD. Under memory pressure, *REMON remote* avoids storage I/O and outperforms disk-based approaches: it reduces latency by 48.3% versus *DuckDB S2D*, and by 33.7% versus *OS S2D*. *REMON S2D* also consistently outperforms *DuckDB S2D*, suggesting that DBMS-layer spilling incurs extra coordination overhead compared to REMON’s page management.

TPC-DS Evaluation. To further evaluate REMON’s performance and versatility, we conducted evaluations using the TPC-DS benchmark.

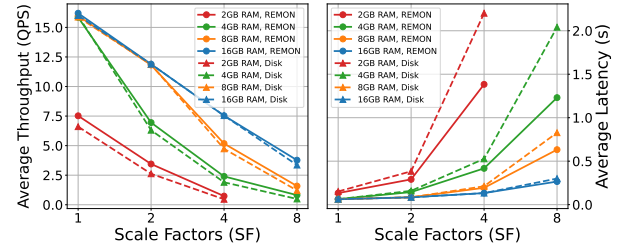


Fig. 11. REMON performance on TPC-DS benchmarks

We ran the benchmark with varying dataset scale factors and local memory limits. Figure 11 compares the average query latencies and average throughput (QPS) of REMON (solid lines) against those of the local disk swap (dashed lines). When the dataset size is within the local RAM capacity, both systems exhibit comparable performance, as no remote memory access is required. However, as the dataset size approaches or exceeds the local memory limit, REMON consistently outperforms local disk swap. This improvement highlights the efficiency of REMON’s remote memory mechanism in minimizing latency under constrained memory conditions.

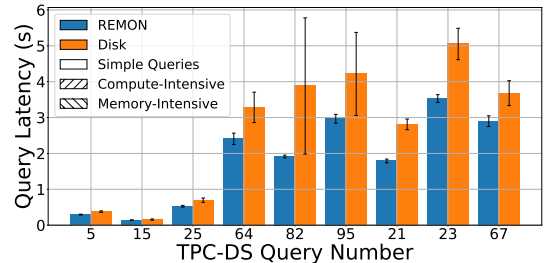


Fig. 12. REMON performance on TPC-DS benchmarks per query

To provide a more detailed analysis, we examined the latency distribution among different types of TPC-DS queries for a dataset scale factor of 8 GB and a local RAM limit of 8 GB. Figure 12 shows that REMON achieves comparable

performance on simple queries and much lower latencies than local disk swap on both compute-intensive and memory-intensive queries.

These results reaffirm REMON’s ability to handle complex analytical workloads, demonstrating its robustness and suitability for scenarios requiring scalable memory management, such as cloud-based decision support systems.

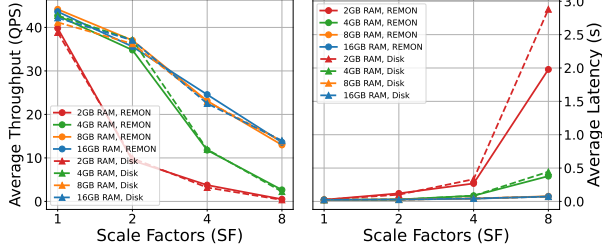


Fig. 13. REMON performance on SSB benchmarks

SSB Evaluation. We also evaluated REMON using the SSB benchmark under varying local RAM constraints and scale factors. As illustrated in Figure 13, the results align with those observed in the TPC-H and TPC-DS benchmarks, showcasing REMON’s consistent performance across workloads. Note that the dataset size and memory requirements during runtime for SSB are lower than those of the other benchmarks. As a result, REMON demonstrates a significant advantage in scenarios with a higher scale-factor-to-RAM ratio. For example, with a scale factor of 8 and a local RAM limit of 2 GB, REMON reduces average latency by approximately 31% versus local disk swap, showcasing its efficiency in constrained memory environments.

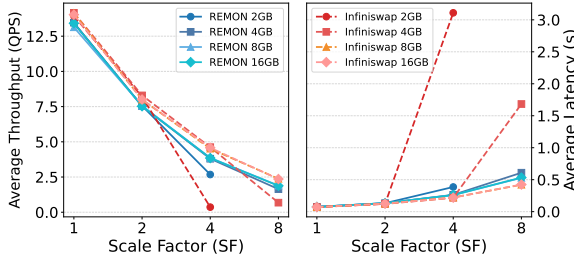


Fig. 14. Infiniswap (RDMA) vs. REMON on CloudLab

RDMA-Based Solutions. We next compared REMON with Infiniswap, an RDMA-based remote-memory system, on the CloudLab platform. We ran DuckDB with TPC-H while varying local RAM and scale factor. Figure 14 shows that Infiniswap generally achieves higher throughput, but REMON remains close: across stable configurations, REMON is about 11% behind Infiniswap in throughput on average. Under the most memory-constrained setting (e.g., RAM=2 GB at SF=4), REMON even exceeds Infiniswap, suggesting that REMON’s user-space paging can handle extreme pressure more gracefully. The degradation of Infiniswap under such settings is likely related to frequent page swapping in the kernel layer. Overall, the results quantify the performance trade-off between RDMA-supported remote memory and REMON’s deployability on commodity networks.

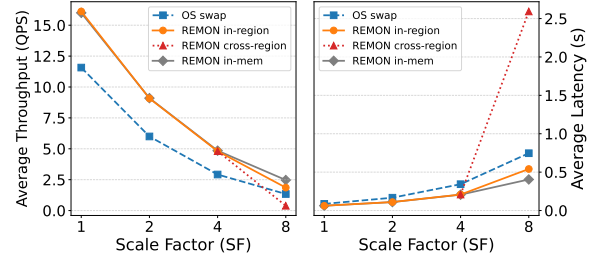


Fig. 15. DuckDB+REMON performance on AWS EC2

Commodity Cloud Deployment. We also deployed DuckDB+REMON on AWS EC2 to validate REMON on commodity cloud networks and to study sensitivity to wide-area latency. We evaluated an in-region (inter-AZ) setup and a cross-region setup under a 4 GB local-RAM limit across scale factors. Figure 15 shows that REMON performs well in-region and consistently outperforms SSD-backed OS swapping, even under high memory pressure (e.g., SF=8). In contrast, the cross-region deployment incurs substantially higher RTT and lower bandwidth, which can negate the benefit of remote memory and lead to worse performance than disk-based swapping. This behavior aligns with the sensitivity study in Figure 9 and reinforces that REMON is best suited for in-region clusters with low RTT and sufficient bandwidth.

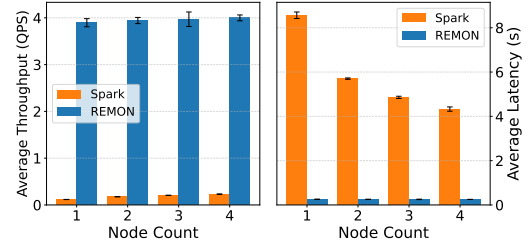


Fig. 16. Spark vs. REMON on the private cloud

Distributed Databases Baseline. We compared REMON with distributed analytics engines using Spark SQL on our private cloud. We ran TPC-H (SF=8) and varied the number of Spark worker nodes, reporting latency and throughput in Figure 16. Spark benefits from additional workers, but remains much slower than DuckDB+REMON. Overall, the comparison suggests that augmenting a single-node engine with an elastic memory layer (REMON) can outperform moving to a distributed engine for analytical workloads.

C. Cost-Performance Analysis

We examined whether REMON’s performance benefits translate into improved cost-performance on commodity cloud platforms. We ran TPC-H (SF=4) and compared five deployment options: (1) *AWS REMON*: DuckDB with REMON remote memory using one compute instance and one memory instance; (2) *AWS SSD*: vanilla DuckDB on a standard instance with SSD-backed OS swapping; (3) *AWS in-mem*: vanilla DuckDB on a single high-memory instance sized to run fully in memory; (4) *Amazon Redshift*: a provisioned

ra3.4xlarge cluster with 1–4 nodes; and (5) *Snowflake*: Small/Medium/Large warehouses on AWS¹.

Figure 17 plots each option with on-demand hourly cost (USD/h) on the x-axis and measured throughput (QPS) on the y-axis. Overall, REMON achieves a favorable cost-performance point: the high-memory instance improves throughput by only 15% while costing 64% more than the REMON remote configuration, and SSD-backed swapping is both slower and more expensive. Redshift and Snowflake are also more costly while delivering lower throughput in each setting, highlighting REMON as a practical and cost-effective alternative for memory-intensive analytical workloads.

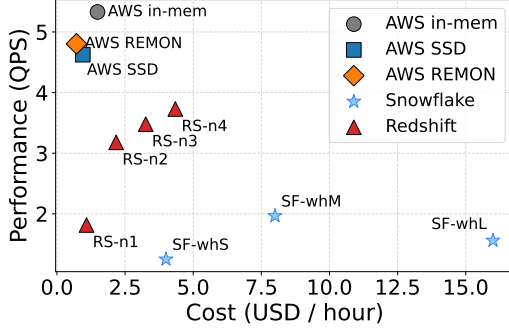


Fig. 17. Cost-performance comparison across alternatives (TPC-H, SF = 4)

VI. DISCUSSIONS

REMON provides an elastic and adaptable disaggregated memory layer over commodity networks. Its user-space design avoids specialized hardware and kernel changes, making it compatible with cloud-native deployments and orchestration platforms like Kubernetes [52]. In practice, benefits materialize under in-region, low-latency, high-bandwidth links; sensitivity to network conditions is an inherent trade-off of networked memory. This elasticity allows the cluster’s memory capacity to be provisioned independently of compute: an operator can scale pooled capacity by adding memory nodes without changing the compute engine. REMON establishes per-client sessions with capacity negotiation and distributes evicted pages across memory nodes within a deployment.

A practical implication is that performance improvements are strongest when local RAM is the primary bottleneck and interconnects are fast. Under higher delays or reduced bandwidth, the advantage over disk-based swapping diminishes. Our experiments span a private cloud, CloudLab, and AWS EC2; behavior under different CPUs, storage, or networking stacks may vary. Workloads here are analytical and predominantly read-intensive; mixed and write-heavy patterns may exhibit different dynamics.

REMON currently does not include fault-tolerant mechanisms for memory node failures. In the event of a remote memory node failure, pages stored on that node would become inaccessible, potentially leading to application errors or

crashes. Future work could explore checkpointing or replication strategies to support graceful recovery and enhance resilience against such failures.

REMON is transparent to applications: a DBMS can continue using its native buffer manager on top of REMON, treating REMON as an external allocator. Optionally, it can provide lightweight hints to guide page placement/eviction. REMON can be further improved with page-management policies that use application hints to complement LRU-based decisions. Examples include (a) operator-aware page grouping (e.g., pinning hash-build inputs during joins and unpinning on probe completion), (b) lightweight prefetch guided by scan ranges and pipeline boundaries, and (c) a minimal hint API for cost-based optimizers to mark hot/cold regions or deadlines. In addition, low-overhead instrumentation (access counters and reuse-distance estimates) can help tune policies offline without changing applications. These policy improvements aim to strengthen performance robustness across diverse workloads without sacrificing deployability.

REMON’s abstraction is also applicable to support memory-bound analytics (e.g., graph traversals and large joins), accelerate data science and model training pipelines by enabling larger in-memory working sets (e.g., larger batches and feature tables), and serve as a capacity tier for in-memory data stores and feature caches. These use cases benefit from transparent memory extension without kernel or hardware changes in commodity clouds.

VII. CONCLUSIONS

REMON expands the memory capacity of database servers by pooling remote memory over commodity TCP/IP. Using a three-tier hierarchy and user-space page movement between local RAM, remote memory, and local disk, REMON outperforms disk-based swapping and compares favorably with the software-based remote-memory baseline R3map and distributed analytics baseline Spark, while remaining close to the RDMA-based approach (Infiniswap) and staying deployable and cost-effective on commodity clouds such as AWS.

In conclusion, REMON offers a lightweight, software-based disaggregated memory solution tailored for in-process database systems like DuckDB. In typical in-region cloud deployments with low latency and high bandwidth, REMON approaches in-memory performance while offering favorable cost efficiency versus provisioning a single high-memory instance.

ACKNOWLEDGEMENT

We acknowledge the contributions of several individuals to early stages of this work. Zongxin Liu developed and implemented the REMON prototype during his engagement with the research group from 2022 to 2024. Bryan Yan conducted early experimental evaluations of REMON. Wing Piu Lee introduced us to R3map and performed initial experiments assessing its performance. This work was in part funded by NSERC and ORF.

¹ Snowflake does not expose the underlying instance types or node counts associated with each warehouse size.

AI-GENERATED CONTENT ACKNOWLEDGEMENT

We use an AI-assisted writing tool (OpenAI ChatGPT) to polish wording and perform sanity checks. The AI tool did not generate novel technical content or algorithms. All final content is authored, edited, and validated by the authors.

REFERENCES

- [1] M. Zwolenski and L. Weatherill, “The digital universe: Rich data and the increasing value of the internet of things,” *Journal of Telecommunications and the Digital Economy*, vol. 2, no. 3, pp. 47–1, 2014.
- [2] M. Zaharia, M. Chowdhury, T. Das, A. Dave, J. Ma, M. McCauly, M. J. Franklin, S. Shenker, and I. Stoica, “Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing,” in *9th USENIX Symposium on Networked Systems Design and Implementation (NSDI 12)*. San Jose, CA: USENIX Association, Apr. 2012, pp. 15–28. [Online]. Available: <https://www.usenix.org/conference/nsdi12/technical-sessions/presentation/zaharia>
- [3] M. Satyanarayanan, “The emergence of edge computing,” *Computer*, vol. 50, no. 1, pp. 30–39, 2017.
- [4] N. T. Friedman and N. Heudecker, “Data hubs, data lakes and data warehouses: how they are different and why they are better together,” *Gartner online at: https://www.gartner.com/doc/reprints*, 2020.
- [5] Y. Chen, S. Alspaugh, and R. Katz, “Interactive analytical processing in big data systems: A cross-industry study of mapreduce workloads,” *arXiv preprint arXiv:1208.4174*, 2012.
- [6] D. Xie, F. Li, B. Yao, G. Li, L. Zhou, and M. Guo, “Simba: Efficient in-memory spatial analytics,” in *Proceedings of the 2016 international conference on management of data*, 2016, pp. 1071–1085.
- [7] K.-L. Tan, Q. Cai, B. C. Ooi, W.-F. Wong, C. Yao, and H. Zhang, “In-memory databases: Challenges and opportunities from software and hardware perspectives,” *ACM Sigmod Record*, vol. 44, no. 2, pp. 35–40, 2015.
- [8] A. S. Foundation, “Apache Spark: A unified analytics engine for large-scale data processing,” <https://spark.apache.org/>, 2023, accessed: April, 2023.
- [9] S. Sanfilippo, *Redis*, Redis Labs, 2009–present, <https://redis.io>.
- [10] Amazon Web Services, “Amazon EC2 on-demand pricing,” <https://aws.amazon.com/ec2/pricing/on-demand/>, 2024.
- [11] —, “Amazon EC2 instance types and regions,” <https://docs.aws.amazon.com/ec2/latest/instancetypes/ec2-instance-regions.html>, 2024.
- [12] —, “Amazon simple storage service (Amazon S3),” <https://aws.amazon.com/s3/>, 2023. [Online]. Available: <https://aws.amazon.com/s3/>
- [13] S. Kotni, A. Nayak, V. Ganapathy, and A. Basu, “Faastlane: Accelerating function-as-a-service workflows,” in *2021 USENIX Annual Technical Conference (USENIX ATC 21)*. USENIX Association, Jul. 2021, pp. 805–820. [Online]. Available: <https://www.usenix.org/conference/atc21/presentation/kotni>
- [14] D. Durner, V. Leis, and T. Neumann, “Exploiting cloud object storage for high-performance analytics,” *Proceedings of the VLDB Endowment*, vol. 16, no. 11, pp. 2769–2782, 2023.
- [15] J. DeBrabant, A. Pavlo, S. Tu, M. Stonebraker, and S. Zdonik, “Anti-caching: A new approach to database management system architecture,” *Proceedings of the VLDB Endowment*, vol. 6, no. 14, pp. 1942–1953, 2013.
- [16] M. Stonebraker, S. Madden, D. J. Abadi, S. Harizopoulos, N. Hachem, and P. Helland, “The end of an architectural era: It’s time for a complete rewrite,” in *Making Databases Work: the Pragmatic Wisdom of Michael Stonebraker*, 2018, pp. 463–489.
- [17] K. Lim, J. Chang, T. Mudge, P. Ranganathan, S. K. Reinhardt, and T. F. Wenisch, “Disaggregated memory for expansion and sharing in blade servers,” *ACM SIGARCH computer architecture news*, vol. 37, no. 3, pp. 267–278, 2009.
- [18] T. Von Eicken, A. Basu, V. Buch, and W. Vogels, “U-Net: A user-level network interface for parallel and distributed computing,” *ACM SIGOPS Operating Systems Review*, vol. 29, no. 5, pp. 40–53, 1995.
- [19] J. Gu, Y. Lee, Y. Zhang, M. Chowdhury, and K. G. Shin, “Efficient memory disaggregation with infiniswap,” in *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*, 2017, pp. 649–667.
- [20] Y. Shan, Y. Huang, Y. Chen, and Y. Zhang, “LegoOS: A disseminated, distributed OS for hardware resource disaggregation,” in *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. Carlsbad, CA: USENIX Association, Oct. 2018, pp. 69–87. [Online]. Available: <https://www.usenix.org/conference/osdi18/presentation/shan>
- [21] Amazon Web Services, “Amazon EC2 P4 instance types,” https://aws.amazon.com/ec2/instance-types/p4/?nc1=h_ls, 2024.
- [22] Z. Guo, Y. Shan, X. Luo, Y. Huang, and Y. Zhang, “Clío: A hardware-software co-designed disaggregated memory system,” in *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2022, pp. 417–433.
- [23] P. X. Gao, A. Narayan, S. Karandikar, J. Carreira, S. Han, R. Agarwal, S. Ratnasamy, and S. Shenker, “Network requirements for resource disaggregation,” in *12th USENIX symposium on operating systems design and implementation (OSDI 16)*, 2016, pp. 249–264.
- [24] Q. Zhang, Y. Cai, X. Chen, S. Angel, A. Chen, S. Han, Y. Liu, and B. T. Loo, “Understanding the effect of data center resource disaggregation on production DBMSs,” *Proceedings of the VLDB Endowment*, vol. 13, no. 9, 2020.
- [25] Amazon Web Services, “Cloud data warehouse - Amazon Redshift - AWS,” <https://aws.amazon.com/redshift/>, [Accessed 19-01-2026].
- [26] B. Dageville, T. Cruanes, M. Zukowski, V. Antonov, A. Avanes, J. Bock, J. Claybaugh, D. Engovatov, M. Hentschel, J. Huang *et al.*, “The Snowflake elastic data warehouse,” in *Proceedings of the 2016 International Conference on Management of Data*, 2016, pp. 215–226.
- [27] DuckDB, “Duckdb,” <https://duckdb.org>, 2023. [Online]. Available: <https://duckdb.org>
- [28] Transaction Processing Performance Council (TPC), “TPC-H: A decision support benchmark,” 1999. [Online]. Available: https://www.tpc.org/tpc_documents_current_versions/pdf/tpc-h_v2.18.0.pdf
- [29] —, “TPC-DS: A decision support benchmark,” 2003. [Online]. Available: https://www.tpc.org/tpc_documents_current_versions/pdf/tpc-ds_v2.8.0.pdf
- [30] P. O’Neil, E. O’Neil, X. Chen, and S. Revilak, “The star schema benchmark and augmented fact table indexing,” in *Performance Evaluation and Benchmarking: First TPC Technology Conference, TPCTC 2009, Lyon, France, August 24-28, 2009, Revised Selected Papers 1*. Springer, 2009, pp. 237–252.
- [31] C. Amza, A. L. Cox, S. Dwarkadas, P. Keleher, H. Lu, R. Rajamony, W. Yu, and W. Zwaenepoel, “Treadmarks: Shared memory computing on networks of workstations,” *Computer*, vol. 29, no. 2, pp. 18–28, 1996.
- [32] P. Keleher, A. L. Cox, and W. Zwaenepoel, “Lazy release consistency for software distributed shared memory,” *ACM SIGARCH Computer Architecture News*, vol. 20, no. 2, pp. 13–21, 1992.
- [33] K. Li and P. Hudak, “Memory coherence in shared virtual memory systems,” *ACM Transactions on Computer Systems (TOCS)*, vol. 7, no. 4, pp. 321–359, 1989.
- [34] R. Wang, J. Wang, S. Idreos, M. T. Özsu, and W. G. Aref, “The case for distributed shared-memory databases with RDMA-enabled memory disaggregation,” *Proc. VLDB Endow.*, vol. 16, no. 1, pp. 15–22, Sep. 2022. [Online]. Available: <https://doi.org/10.14778/3561261.3561263>
- [35] T. Ziegler, C. Binnig, and V. Leis, “Scalestore: A fast and cost-efficient storage engine using DRAM, NVMe, and RDMA,” in *Proceedings of the 2022 International Conference on Management of Data*, 2022, pp. 685–699.
- [36] W. Cao and L. Liu, “Dynamic and transparent memory sharing for accelerating big data analytics workloads in virtualized cloud,” in *2018 IEEE International Conference on Big Data (Big Data)*. IEEE, 2018, pp. 191–200.
- [37] W. Cao, Z. Liu, P. Wang, S. Chen, C. Zhu, S. Zheng, Y. Wang, and G. Ma, “Polarfs: an ultra-low latency and failure resilient distributed file system for shared storage cloud database,” *Proceedings of the VLDB Endowment*, vol. 11, no. 12, pp. 1849–1862, 2018.
- [38] M. K. Aguilera, N. Amit, I. Calciu, X. Deguillard, J. Gandhi, S. Novakovic, A. Ramanathan, P. Subrahmanyam, L. Suresh, K. Tati *et al.*, “Remote regions: a simple abstraction for remote memory,” in *2018 USENIX Annual Technical Conference (USENIX ATC 18)*, 2018, pp. 775–787.
- [39] Z. Ruan, M. Schwarzkopf, M. K. Aguilera, and A. Belay, “AIFM: High-performance, application-integrated far memory,” in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, Nov. 2020, pp. 315–332. [Online]. Available: <https://www.usenix.org/conference/osdi20/presentation/ruan>

- [40] pojntfx, “Remote mmap: High-performance remote memory region mounts and migrations in user space.” 2024. [Online]. Available: <https://github.com/pojntfx/r3map>
- [41] J. Ousterhout, P. Agrawal, D. Erickson, C. Kozyrakis, J. Leverich, D. Mazières, S. Mitra, A. Narayanan, D. Ongaro, G. Parulkar *et al.*, “The case for RAMCloud,” *Communications of the ACM*, vol. 54, no. 7, pp. 121–130, 2011.
- [42] V. Shrivastav, A. Valadarsky, H. Ballani, P. Costa, K. S. Lee, H. Wang, R. Agarwal, and H. Weatherspoon, “Shoal: A network architecture for disaggregated racks,” in *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*, 2019, pp. 255–270.
- [43] R. L. Mattson, J. Gecsei, D. R. Slutz, and I. L. Traiger, “Evaluation techniques for storage hierarchies,” *IBM Systems journal*, vol. 9, no. 2, pp. 78–117, 1970.
- [44] P. R. Wilson, M. S. Johnstone, M. Neely, and D. Boles, “Dynamic storage allocation: A survey and critical review,” in *Memory Management, International Workshop IWMM 95, Kinross, UK, September 27–29, 1995, Proceedings*, ser. Lecture Notes in Computer Science, vol. 986. Springer, 1995, pp. 1–116. [Online]. Available: https://doi.org/10.1007/3-540-60368-9_19
- [45] E. D. Berger, K. S. McKinley, R. D. Blumofe, and P. R. Wilson, “Hoard: A scalable memory allocator for multithreaded applications,” in *Proceedings of the 9th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS IX)*. ACM Press, 2000, pp. 117–128. [Online]. Available: <https://doi.org/10.1145/378993.379232>
- [46] J. Evans, “A scalable concurrent `malloc(3)` implementation for FreeBSD,” BSDCan 2006 slides, 2006, accessed 2026-01-22. [Online]. Available: <https://people.freebsd.org/~jasone/jemalloc/bdcan2006/jemalloc.pdf>
- [47] D. Lea, “A memory allocator,” Web page, 2000, last modified April 4, 2000; accessed 2026-01-22. [Online]. Available: <https://gee.cs.oswego.edu/dl/html/malloc.html>
- [48] M. Armbrust, R. S. Xin, C. Lian, Y. Huai, D. Liu, J. K. Bradley, X. Meng, T. Kaftan, M. J. Franklin, A. Ghodsi *et al.*, “Spark SQL: Relational data processing in Spark,” in *Proceedings of the 2015 ACM SIGMOD international conference on management of data*, 2015, pp. 1383–1394.
- [49] D. Duplyakin, R. Ricci, A. Maricq, G. Wong, J. Duerig, E. Eide, L. Stoller, M. Hibler, D. Johnson, K. Webb, A. Akella, K. Wang, G. Ricart, L. Landweber, C. Elliott, M. Zink, E. Cecchet, S. Kar, and P. Mishra, “The design and operation of CloudLab,” in *Proceedings of the USENIX Annual Technical Conference (ATC)*, Jul. 2019, pp. 1–14. [Online]. Available: <https://www.flux.utah.edu/paper/duplyakin-atc19>
- [50] Netem, “Netem manual,” <https://man7.org/linux/man-pages/man8/tc-netem.8.html>, n.d.
- [51] H. Iqbal, A. Singh, and M. Shahzad, “Characterizing the availability and latency in AWS network from the perspective of tenants,” *IEEE/ACM Transactions on Networking*, vol. 30, no. 4, pp. 1554–1568, 2022.
- [52] B. Burns, J. Beda, K. Hightower, and L. Evenson, *Kubernetes: up and running: dive into the future of infrastructure*. ” O’Reilly Media, Inc.”, 2022.